Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программная инженерия»

Лабораторная работа № 2

по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ алгоритмов умножения матриц

Студент Понарин А. А.

Группа ИУ7-55Б

Преподаватели Волкова Л. Л., Строганов Д. В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Аналитическая часть **4**
1.1 Постановка задачи 4
1.2 Классический алгоритм 4
1.3 Алгоритм Винограда 4
1.4 Оптимизированный алгоритм Винограда 5

2 Конструкторская часть **6**
2.1 Схемы алгоритмов 6
2.2 Модель трудоёмкости 11
2.3 Трудоёмкость классического алгоритма 12
2.4 Трудоёмкость алгоритма Винограда 12
2.5 Трудоёмкость оптимизированного варианта 13
2.6 Оценка памяти 13
2.7 Требования к программе 14

3 Технологическая часть **15**
3.1 Средства реализации 15
3.2 Реализация алгоритмов 15
3.3 Измерение процессорного времени 17
3.4 Тестирование 18

4 Исследовательская часть **20**
4.1 Условия проведения эксперимента 20
4.2 Методика массированных замеров 20
4.3 Результаты замеров 21
4.4 Сопоставление теории и эксперимента 23

ЗАКЛЮЧЕНИЕ **25**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **26**

ВВЕДЕНИЕ

Цель работы — провести сравнительный анализ алгоритмов умножения матриц.

Для достижения поставленной цели необходимо решить следующие задачи:

1. описать классический алгоритм умножения матриц, алгоритм Винограда и его оптимизированный вариант;
2. разработать схемы алгоритмов и оценить их трудоёмкость и затраты памяти;
3. реализовать алгоритмы на языке C++ и проверить их корректность;
4. выполнить массированные замеры процессорного времени при одинаковых условиях;
5. сравнить результаты для чётных и нечётных размеров и установить, согласуются ли замеры с теоретическими оценками.

Оформление отчёта выполнено с учётом ГОСТ 7.32–2017 [1].

1 Аналитическая часть

1.1 Постановка задачи

Даны матрицы $A \in \mathbb{Z}^{m\times n}$ и $B \in \mathbb{Z}^{n\times q}$. Требуется получить матрицу $C = AB \in \mathbb{Z}^{m\times q}$:

$$c_{ij} = \sum_{k=0}^{n-1} a_{ik} b_{kj}, \quad 0 \le i < m, \quad 0 \le j < q. \tag{1.1}$$

Число столбцов первой матрицы должно совпадать с числом строк второй. В программе размеры ограничены диапазоном от 1 до 2000, элементы — диапазоном от -10^6 до 10^6. Вычисления выполняются в знаковых 64-разрядных целых числах. Для этих ограничений переполнение отсутствует: модуль результата не превосходит $2 \cdot 10^{15}$, а грубая верхняя граница промежуточной суммы Винограда составляет $6 \cdot 10^{15} < 2^{63} - 1$.

1.2 Классический алгоритм

Матрица результата заполняется нулями. Для каждой пары индексов i, j во внутреннем цикле по k к c_{ij} прибавляется произведение $a_{ik}b_{kj}$. Порядок циклов реализации — i, j, k. Алгоритм применим к прямоугольным матрицам; значения элементов не влияют на число итераций.

1.3 Алгоритм Винограда

Метод Винограда основан на преобразовании скалярного произведения [2]. Для двух соседних слагаемых справедливо тождество

$$xy + uv = (x + v)(u + y) - xu - yv. \tag{1.2}$$

Положим $d = \lfloor n/2 \rfloor$, $r = n \bmod 2$. Вычислим вспомогательные множители:

$$R_i = \sum_{k=0}^{d-1} a_{i,2k} a_{i,2k+1}, \qquad H_j = \sum_{k=0}^{d-1} b_{2k,j} b_{2k+1,j}. \tag{1.3}$$

Тогда произведение определяется выражением

$$c_{ij} = -R_i - H_j + \sum_{k=0}^{d-1}(a_{i,2k} + b_{2k+1,j}) \cdot \\ \cdot (a_{i,2k+1} + b_{2k,j}) + r\, a_{i,n-1} b_{n-1,j}. \tag{1.4}$$

Последнее слагаемое вычисляется только при нечётном n. Для $n = 1$ суммы пар пусты, и результат целиком даёт коррекция. Множитель R_i используется во всех столбцах строки, а H_j — во всех строках столбца. За счёт этого число умножений сокращается.

1.4 Оптимизированный алгоритм Винограда

Используется та же формула (1.4), но изменена организация вычислений:

1. чётная граница $e = n - n \bmod 2$ и признак нечётности вычисляются один раз;
2. индекс внутреннего цикла увеличивается на 2, обращения $2k$ заменены обращениями k;
3. множители и элементы результата накапливаются в локальных скалярных переменных; запись в массив выполняется после накопления;
4. коррекция нечётного размера включена в вычисление текущего элемента, отдельный проход по результату не нужен.

Это оптимизация реализации; арифметическая формула и асимптотический порядок трудоёмкости сохраняются.

Вывод

Рассмотрены три алгоритма решения одной задачи. Алгоритм Винограда уменьшает число умножений ценой дополнительных сложений и двух массивов множителей. Нечётное число общих измерений требует коррекции.

2 Конструкторская часть

2.1 Схемы алгоритмов

На рисунках 1–6 приведены схемы алгоритмов. Парные блоки с одинаковой буквой обозначают границы цикла; при пустом диапазоне тело пропускается. Входные данные проверяются до вычислений.

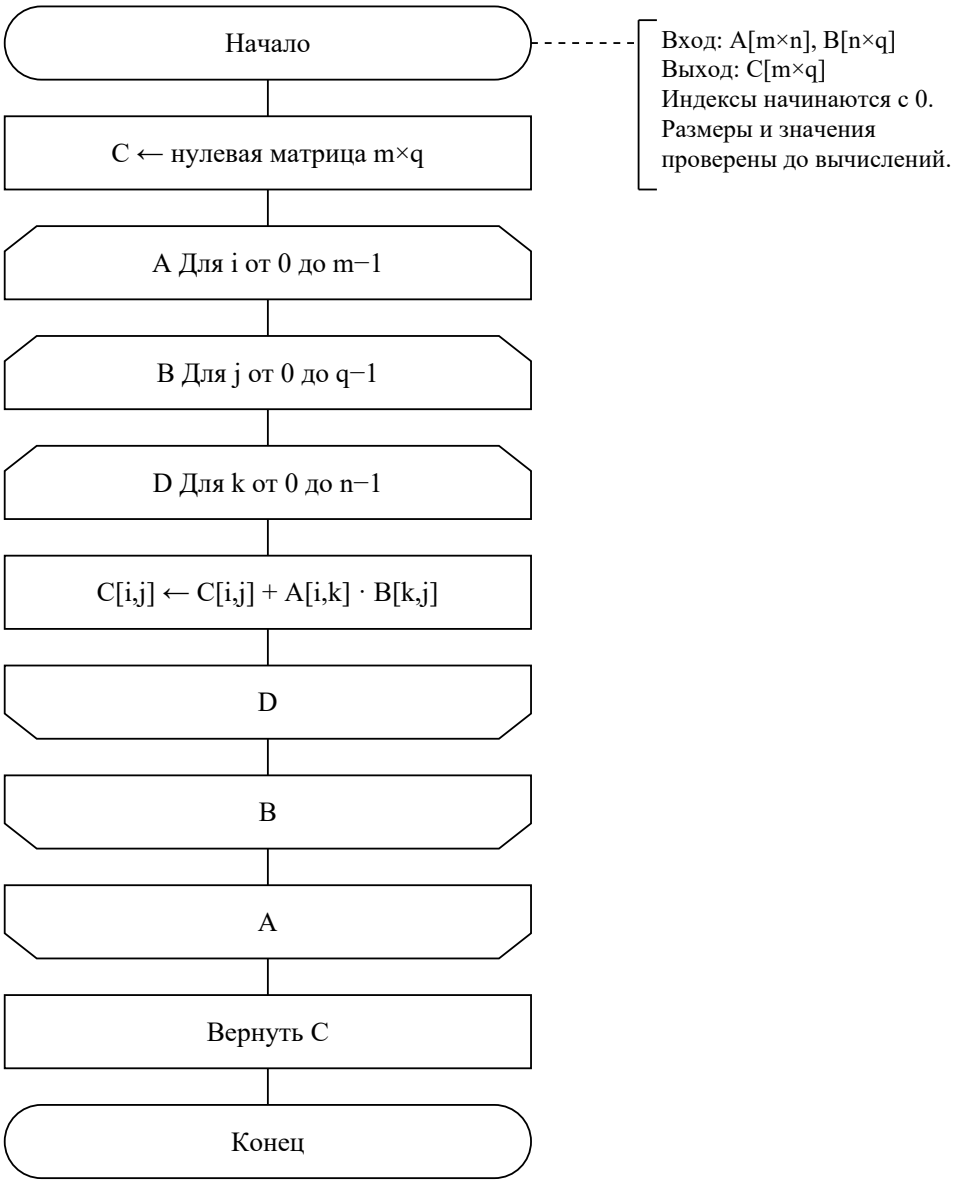


Рисунок 1 – Классический алгоритм умножения матриц

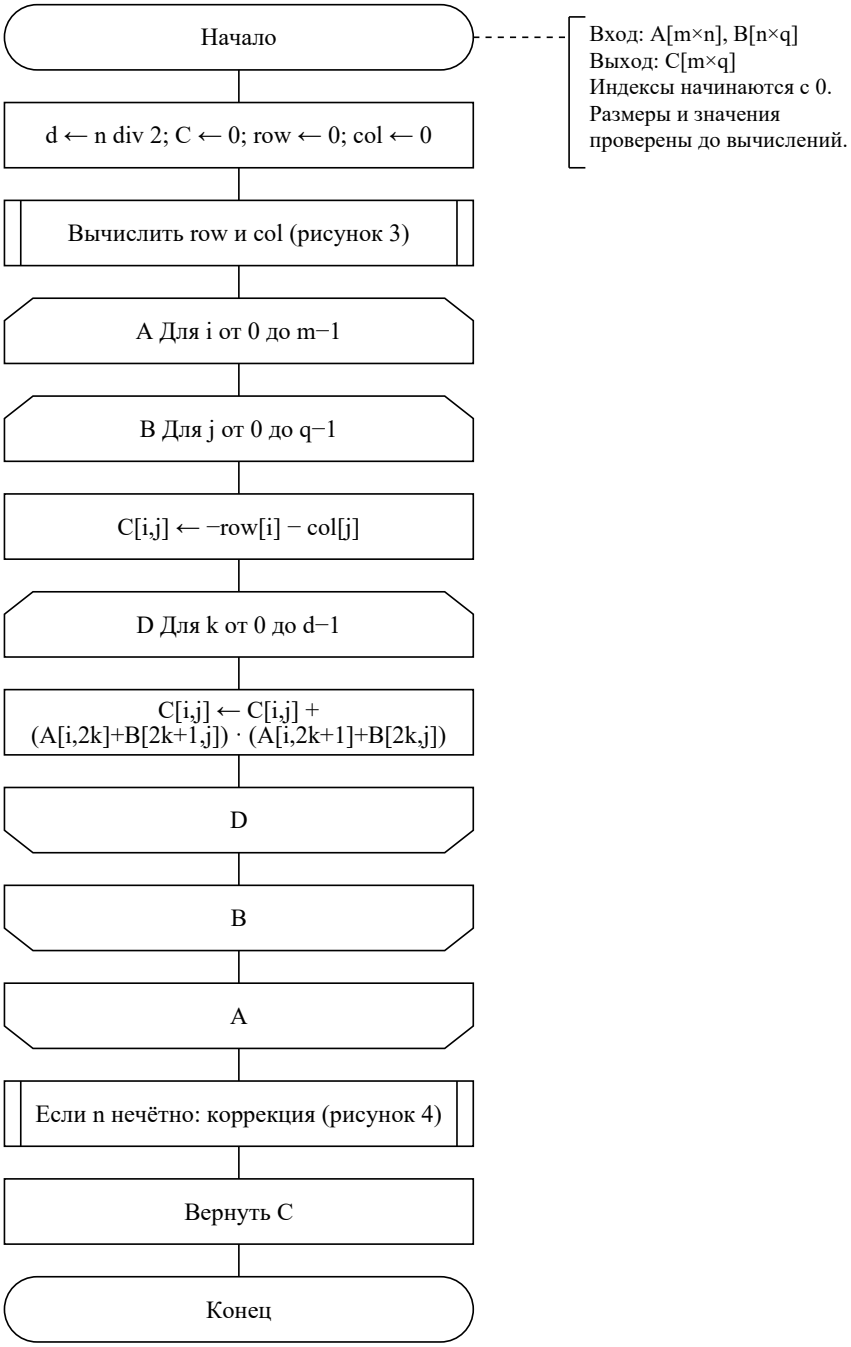


Рисунок 2 – Алгоритм Винограда

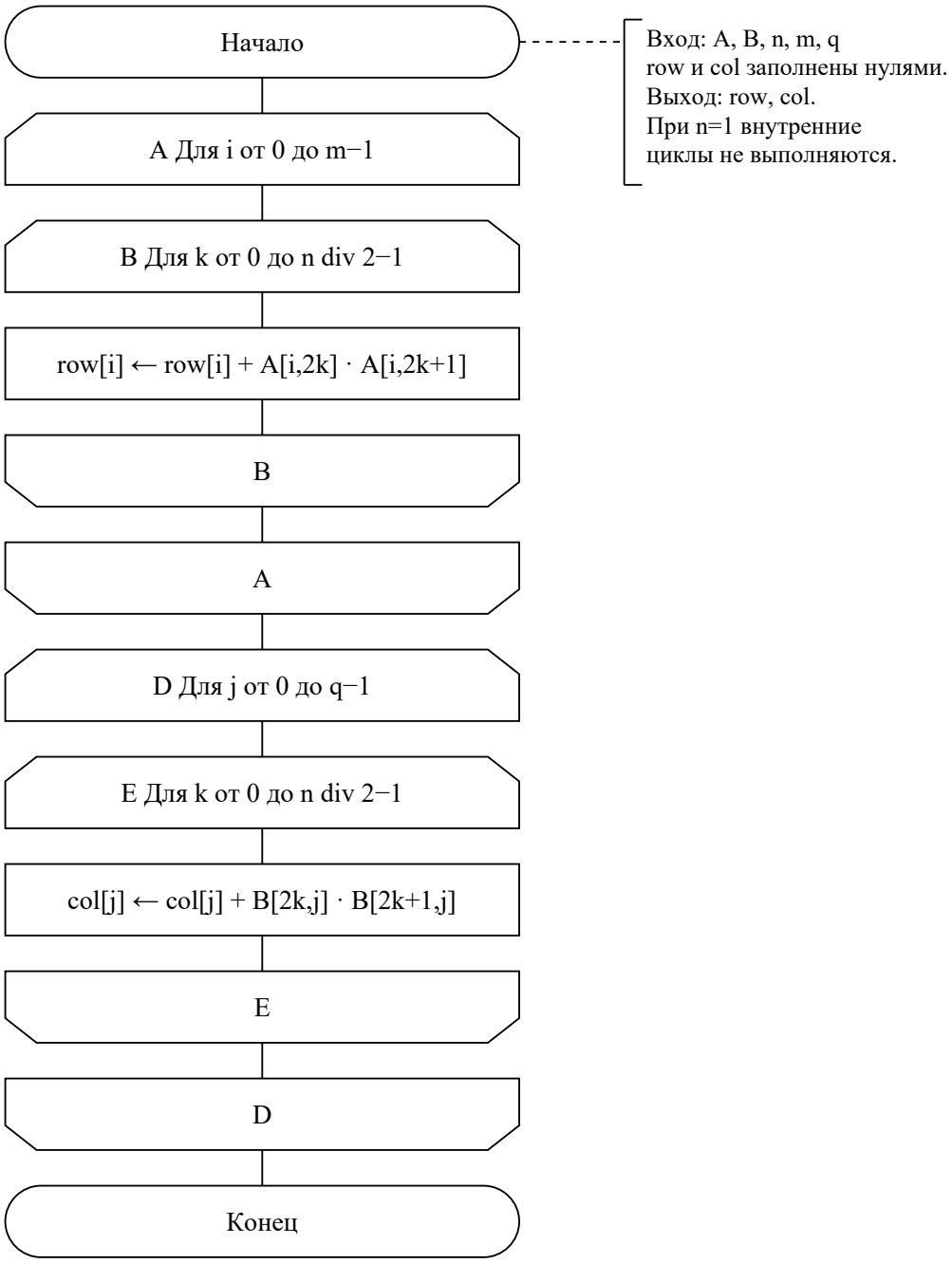


Рисунок 3 – Вычисление множителей алгоритма Винограда

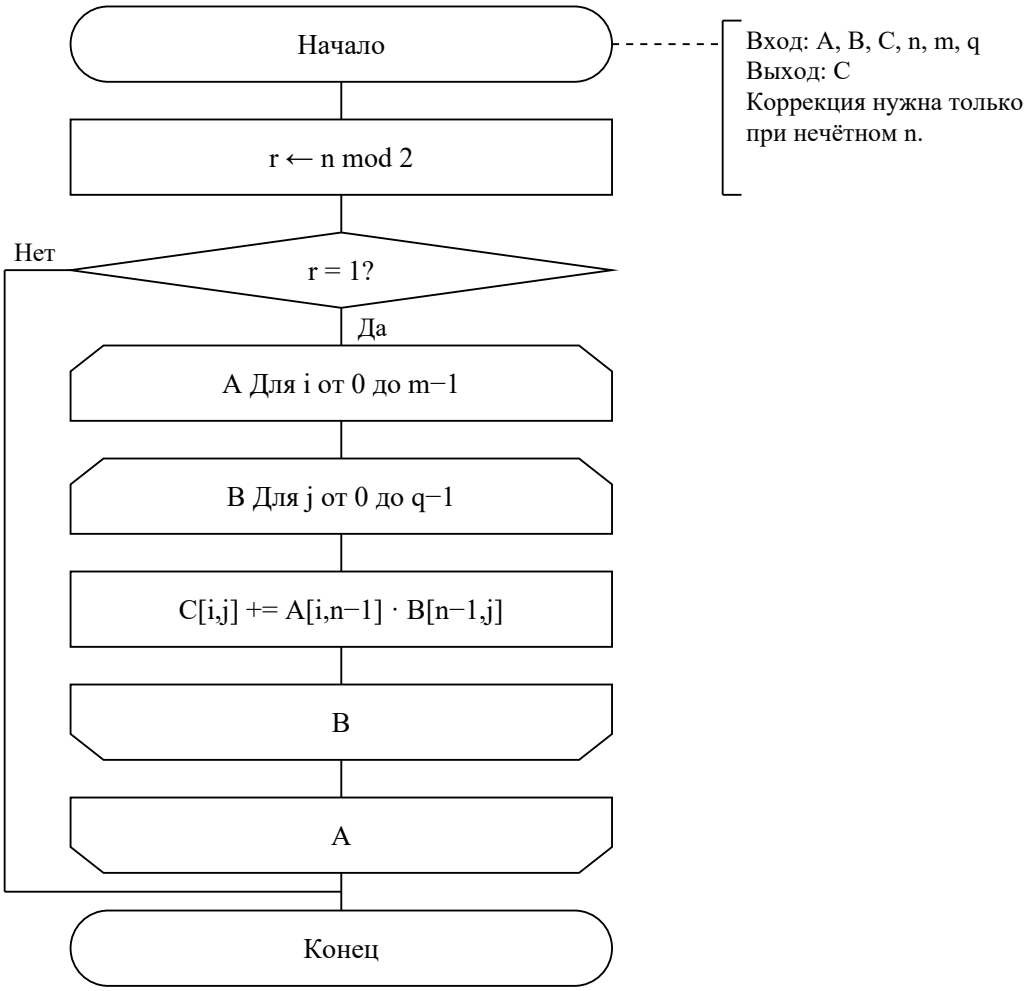


Рисунок 4 – Коррекция результата при нечётном числе общих измерений

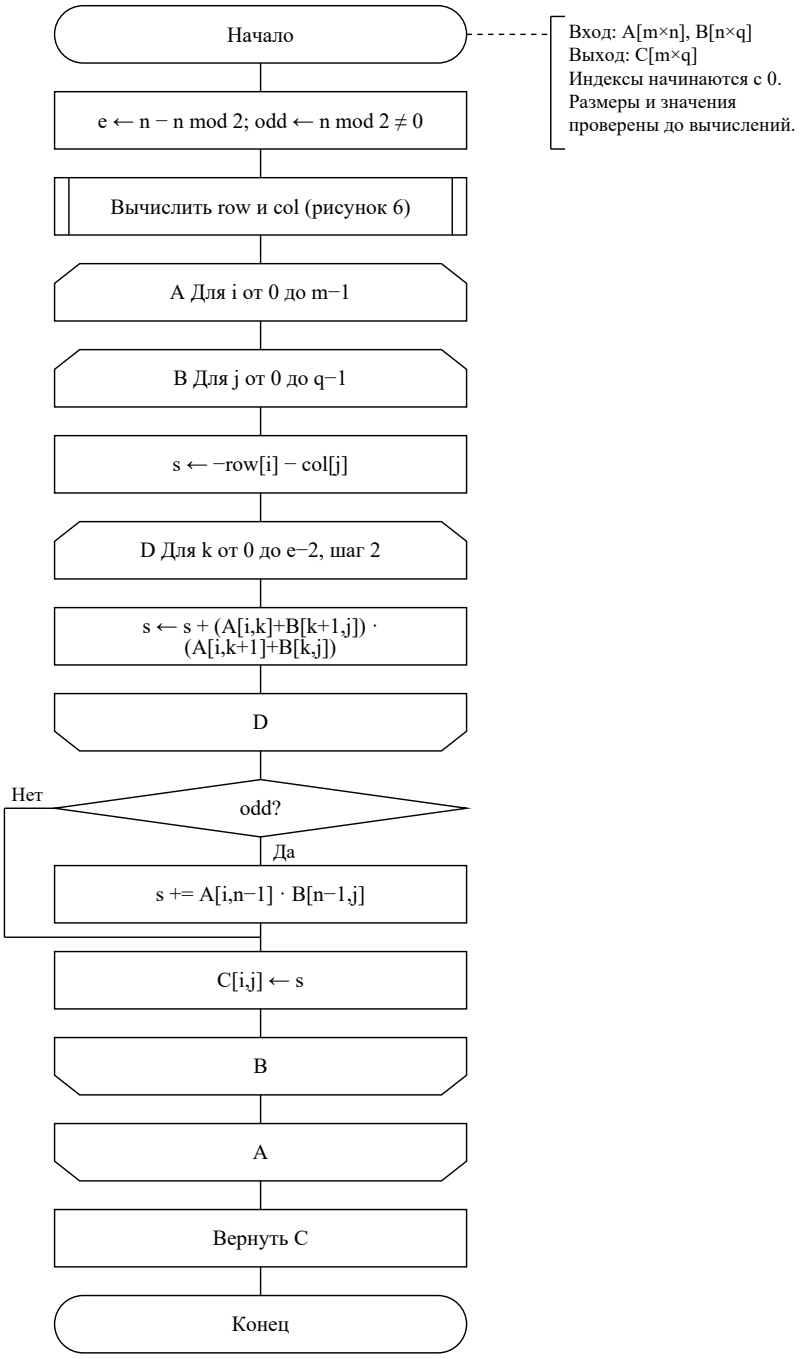


Рисунок 5 – Оптимизированный алгоритм Винограда

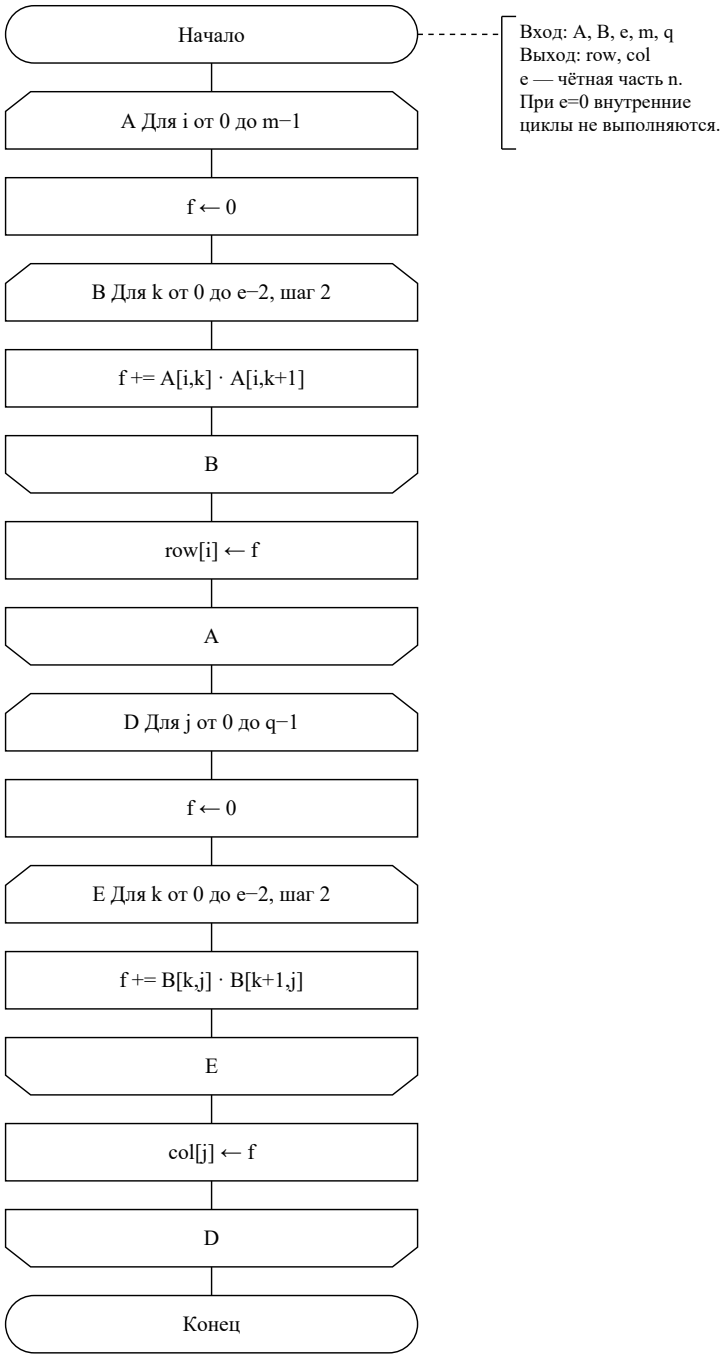


Рисунок 6 – Оптимизированное вычисление множителей

2.2 Модель трудоёмкости

Обозначим через M число умножений элементов матриц, через S — число сложений, вычитаний и унарных изменений знака элементов. Сложению и вычитанию приписывается стоимость α, умножению — μ, где $\alpha, \mu > 0$. Арифметическая трудоёмкость равна

$$F = \alpha S + \mu M. \tag{2.1}$$

Начальное заполнение массива нулями, проверки размеров и значений, операции над индексами, управление циклами и выделение памяти в этой модели не считаются арифметикой элементов матриц. Их влияние на полный алгоритм оценивается отдельно. Модель не отождествляется с количеством машинных инструкций или временем в секундах.

2.3 Трудоёмкость классического алгоритма

Тело внутреннего цикла выполняется mnq раз и содержит одно умножение и одно сложение с текущей суммой, включая первое сложение с нулём. Поэтому

$$M_{\text{кл}} = mnq, \qquad S_{\text{кл}} = mnq, \qquad F_{\text{кл}} = (\alpha + \mu)mnq. \tag{2.2}$$

Для квадратных матриц порядка N получаем $F_{\text{кл}} = (\alpha + \mu)N^3$. Организация циклов не зависит от значений элементов: при фиксированных размерах лучший и худший случаи совпадают по числу операций.

2.4 Трудоёмкость алгоритма Винограда

Вычисление множителей требует $(m+q)d$ умножений и столько же сложений. На каждый элемент результата приходятся две операции для $-R_i - H_j$, d умножений и $3d$ сложений в основном цикле. При нечётном n дополнительно выполняются одно умножение и одно сложение на элемент. Следовательно,

$$M_{\text{В}} = (m+q)d + mq(d+r), \tag{2.3}$$

$$S_{\text{В}} = (m+q)d + mq(3d+2+r), \tag{2.4}$$

$$\begin{aligned} F_{\text{В}} = {} & (\alpha+\mu)(m+q)d + \\ & + mq\big((3\alpha+\mu)d + 2\alpha + (\alpha+\mu)r\big). \end{aligned} \tag{2.5}$$

Для квадратных матриц и чётного N формула принимает вид

$$F_{\text{В}} = \frac{3\alpha+\mu}{2}N^3 + (3\alpha+\mu)N^2. \tag{2.6}$$

При нечётном N, с учётом $d = (N-1)/2$, имеем

$$F_{\text{В}} = \frac{3\alpha+\mu}{2}N^3 + \frac{5\alpha+3\mu}{2}N^2 - (\alpha+\mu)N. \tag{2.7}$$

Для фиксированных размеров лучший и худший случаи также совпадают по числу операций. При одинаковых m, q, d переход от $n = 2d$ к $n = 2d + 1$ добавляет mq умножений и mq сложений.

2.5 Трудоёмкость оптимизированного варианта

Число арифметических операций над элементами совпадает с алгоритмом Винограда: $M_{\text{опт}} = M_{\text{В}}$, $S_{\text{опт}} = S_{\text{В}}$. Изменения относятся к индексам и обращениям к памяти. В исходном варианте выражение $2k$ встречается в телах циклов $(2m + 2q + 4mq)d$ раз, в оптимизированном — ни разу. Это количество записанных выражений в исходном коде, а не гарантия такого же числа машинных умножений: компилятор может преобразовать их в обоих вариантах.

Число записей в C в основном вычислении уменьшается с $mq(d+1)$ до mq; коррекция нечётного случая не требует отдельного прохода. Множители также накапливаются в скалярах. Это объясняет возможность ускорения при одинаковой арифметической трудоёмкости.

Все реализации дополнительно проверяют элементы за $O(mn + nq)$, инициализируют результат за $O(mq)$ и выполняют служебные операции циклов. Полная трудоёмкость каждого алгоритма составляет $\Theta(mnq)$ для положительных размеров; для квадратных матриц — $\Theta(N^3)$.

Если положить $\alpha = \mu = 1$, то для чётного N получаются $F_{\text{кл}} = 2N^3$ и $F_{\text{В}} = 2N^3 + 4N^2$. Следовательно, само уменьшение числа умножений не доказывает ускорения в модели равных стоимостей. При $\mu > \alpha$ ведущий коэффициент Винограда меньше. Реальное время зависит также от компилятора и памяти и требует измерения.

2.6 Оценка памяти

Матрицы хранятся построчно в одномерных массивах элементов типа `int64_t`. Полезные данные входа занимают $8(mn + nq)$ байт, результата — $8mq$ байт. Дополнительные массивы Винограда занимают $8(m + q)$ байт. Оценки не включают постоянные накладные расходы объектов контейнеров и распределителя памяти.

Таблица 1 – Сравнение трудоёмкости и дополнительной памяти

Алгоритм	Время, квадратные матрицы	Память без входа и выхода
Классический	$\Theta(N^3)$	$O(1)$
Винограда	$\Theta(N^3)$	$8(m+q)+O(1)$ байт
Оптимизированный	$\Theta(N^3)$	$8(m+q)+O(1)$ байт

При учёте результата дополнительная память составляет $\Theta(mq)$ для классического алгоритма и $\Theta(mq+m+q)$ для вариантов Винограда. Для квадратных матриц все три оценки равны $\Theta(N^2)$.

2.7 Требования к программе

Программа должна обеспечивать ручной ввод и генерацию матриц, выбор любого из трёх алгоритмов, получение произведения прямоугольных матриц, проверку диапазонов и повтор ввода при ошибке. Отдельные режимы должны выполнять автоматические тесты и воспроизводимые замеры с сохранением результатов в CSV. Вычисления выполняются последовательно, в одном потоке.

Вывод

Разработаны схемы алгоритмов, получены формулы арифметической трудоёмкости и оценки памяти. Оптимизации не меняют кубический порядок; для их практической оценки необходим эксперимент.

3 Технологическая часть

3.1 Средства реализации

Использован язык C++17 и компилятор GCC 14.2.0 (MSYS2 UCRT64). Из стандартной библиотеки применены контейнер `std::vector`, генератор `std::mt19937`, средства ввода-вывода и работы с файлами. Для замеров в Windows выбрана функция `GetProcessTimes` [3].

В файлах `algorithms.hpp` и `algorithms.cpp` определены матрица, проверка допустимости данных и три алгоритма. В `main.cpp` размещены меню, тесты и измерительный стенд. Файл `CMakeLists.txt` задаёт сборку и тест CTest. Сборка измеряемой версии:

Листинг 1 – Команда сборки программы

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic main.cpp algorithms.cpp -o
   matrix_lab.exe
```

3.2 Реализация алгоритмов

В листингах 2–4 приведён код из поставляемого файла реализации. Функция `validate` проверяет совместимость размеров, структуру хранения и допустимость элементов; одинаковая проверка присутствует во всех трёх алгоритмах.

Листинг 2 – Классический алгоритм

```
Matrix classical(const Matrix& a, const Matrix& b) {
    validate(a, b);
    const auto m = a.rows, n = a.cols, q = b.cols;
    Matrix c(m, q);
    for (std::size_t i = 0; i < m; ++i)
        for (std::size_t j = 0; j < q; ++j)
            for (std::size_t k = 0; k < n; ++k)
                c(i, j) += a(i, k) * b(k, j);
    return c;
}
```

Листинг 3 – Алгоритм Винограда

```
Matrix winograd(const Matrix& a, const Matrix& b) {
    validate(a, b);
    const auto m = a.rows, n = a.cols, q = b.cols;
    Matrix c(m, q);
    std::vector<Value> row(m, 0), col(q, 0);
    for (std::size_t i = 0; i < m; ++i)
        for (std::size_t k = 0; k < n / 2; ++k)
            row[i] = row[i] + a(i, 2*k) * a(i, 2*k+1);
    for (std::size_t j = 0; j < q; ++j)
        for (std::size_t k = 0; k < n / 2; ++k)
            col[j] = col[j] + b(2*k, j) * b(2*k+1, j);
    for (std::size_t i = 0; i < m; ++i)
        for (std::size_t j = 0; j < q; ++j) {
            c(i, j) = -row[i] - col[j];
            for (std::size_t k = 0; k < n / 2; ++k)
                c(i, j) = c(i, j) +
                    (a(i, 2*k) + b(2*k+1, j)) *
                    (a(i, 2*k+1) + b(2*k, j));
        }
    if (n % 2 != 0)
        for (std::size_t i = 0; i < m; ++i)
            for (std::size_t j = 0; j < q; ++j)
                c(i, j) += a(i, n-1) * b(n-1, j);
    return c;
}
```

Листинг 4 – Оптимизированный алгоритм Винограда

```
Matrix optimized(const Matrix& a, const Matrix& b) {
    validate(a, b);
    const auto m = a.rows, n = a.cols, q = b.cols;
    const auto even = n - n % 2;
    const bool odd = n % 2 != 0;
    Matrix c(m, q);
    std::vector<Value> row(m, 0), col(q, 0);
    for (std::size_t i = 0; i < m; ++i) {
        Value factor = 0;
        for (std::size_t k = 0; k < even; k += 2)
            factor += a(i, k) * a(i, k+1);
        row[i] = factor;
    }
    for (std::size_t j = 0; j < q; ++j) {
        Value factor = 0;
        for (std::size_t k = 0; k < even; k += 2)
            factor += b(k, j) * b(k+1, j);
        col[j] = factor;
    }
    for (std::size_t i = 0; i < m; ++i)
        for (std::size_t j = 0; j < q; ++j) {
            Value sum = -row[i] - col[j];
            for (std::size_t k = 0; k < even; k += 2)
                sum += (a(i, k) + b(k+1, j)) *
                       (a(i, k+1) + b(k, j));
            if (odd) sum += a(i, n-1) * b(n-1, j);
            c(i, j) = sum;
        }
    return c;
}
```

3.3 Измерение процессорного времени

Реальное время — длительность интервала между началом и концом операции по часам; оно включает ожидание и периоды, когда процесс не исполняется. Процессорное время учитывает выполнение потоков процесса на процессоре. Для однопоточной программы оно исключает время вытеснения другими процессами, но не устраняет влияние частоты процессора и подсистемы памяти.

По официальной документации Microsoft, `GetProcessTimes` возвращает накопленное время исполнения потоков процесса в пользовательском режиме и режиме ядра; значения представлены в 100-наносекундных

единицах [3]. Их сумма используется как процессорное время. Единица представления не означает фактическую точность таймера в 100 нс.

Листинг 5 – Получение процессорного времени процесса

```
double cpu_seconds() {
#ifdef _WIN32
    FILETIME creation, exit, kernel, user;
    if (!GetProcessTimes(GetCurrentProcess(), &creation, &exit, &kernel, &
   user))
        throw std::runtime_error("GetProcessTimes failed");
    auto ticks = [](FILETIME t) {
        return (std::uint64_t(t.dwHighDateTime) << 32) | t.dwLowDateTime;
    };
    return static_cast<double>(ticks(kernel) + ticks(user)) * 1e-7;
#else
    timespec ts{};
    if (clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &ts) != 0)
        throw std::runtime_error("clock_gettime failed");
    return static_cast<double>(ts.tv_sec) + ts.tv_nsec * 1e-9;
#endif
}
```

Значения времени чтения CSV, генерации входных матриц и проверки результатов в измеряемый интервал не входят. В него входят вызов алгоритма, проверка допустимости матриц, выделение и освобождение памяти результата и множителей, вычисление произведения и запись одного элемента в наблюдаемую переменную. Это измерение полной функции, а не изолированного арифметического ядра.

3.4 Тестирование

Автоматические проверки сопоставляют результат всех трёх реализаций с ожидаемым произведением. Для случайных матриц используется отдельная реализация по сумме внешних произведений с порядком циклов k, i, j. В таблице 2 приведены группы проверок.

Таблица 2 – Функциональные проверки

№	Данные / условие	Ожидаемый результат
1	$A = \left(\begin{smallmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{smallmatrix}\right)$, $B = \left(\begin{smallmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{smallmatrix}\right)$	$\left(\begin{smallmatrix} 58 & 64 \\ 139 & 154 \end{smallmatrix}\right)$
2	$A = (-3)$, $B = (7)$	(-21)
3	Нулевые матрицы 3×4 и 4×2	Нулевая 3×2
4–5	AI и IA, A размера 3×3	A
6	$(1, -2, 3) \cdot (4, 5, -6)^T$	(-24)
7	$(10^6, -10^6) \cdot (10^6, -10^6)^T$	$(2 \cdot 10^{12})$
8	1000 случайных наборов, $m, n, q \in [1, 10]$	Совпадение с эталоном
9	Нулевой размер и размер 2001	Исключение
10	Несовместимые размеры, нарушенное хранение, элемент $10^6 + 1$	Исключение в каждом алгоритме

Команда `matrix_lab.exe --test` успешно завершила 1018 тестовых случаев. Семь известных произведений и 1000 случайных наборов проверены во всех трёх алгоритмах; 11 проверок относятся к отклонению недопустимых данных. Начальное значение генератора — 20261023. Дополнительно проверены ручной ввод и восстановление после ошибочного ввода в меню.

Вывод

Реализованы три алгоритма, интерактивный режим, автоматические проверки и измерительный стенд. Тесты подтвердили корректность на приведённых наборах данных; для замеров используется процессорное время.

4 Исследовательская часть

4.1 Условия проведения эксперимента

Замеры выполнены 23.10.2026 на компьютере с процессором Intel Core i3-1115G4 (номинальная частота 3,00 ГГц, 2 ядра и 4 логических процессора), под управлением Windows, сборка 26200. Использована одна и та же программа, собранная GCC 14.2.0 с ключами `-std=c++17 -O2`, без многопоточности и без оптимизации при компоновке.

Исследуются квадратные матрицы порядков 32, 33, 64, 65, 96, 97, 128, 129, 192, 193, 256, 257, 384, 385, 512, 513. Для каждого порядка одна пара матриц генерируется до замеров; она одинакова для всех алгоритмов и всех повторений. Элементы выбираются из целых чисел от -10 до 10, начальное значение генератора равно 20261023.

4.2 Методика массированных замеров

Для каждой пары матриц выполняется предварительный вызов каждого алгоритма с проверкой результата. Далее число повторений K выбирается отдельно для каждого алгоритма: начиная с $K = 10$, оно удваивается, пока длительность пробного блока не составит хотя бы 0,25 с процессорного времени.

Затем выполняются пять серий. В каждой серии один и тот же алгоритм вызывается K раз подряд на неизменных входных данных; среднее время одного вызова определяется формулой

$$t_s = \frac{T_{\text{CPU,end}} - T_{\text{CPU,start}}}{K}. \tag{4.1}$$

Итог — медиана пяти t_s. Порядок алгоритмов циклически сдвигается между сериями. Значение K может различаться, но правило его выбора и состав измеряемой операции одинаковы. Время генерации, вывода и сравнения матриц исключено. Один элемент каждого результата записывается в `volatile`-переменную; все реализации размещены в отдельной единице трансляции, что препятствует удалению вычислений компилятором.

Исходные результаты находятся в `results/raw.csv`: размер, алгоритм, номер серии, K, суммарное процессорное время и среднее время вызо-

ва. В `results/summary.csv` приведены медиана, минимум и максимум средних времён пяти серий. Различия между сериями характеризуют разброс, а не доверительный интервал.

4.3 Результаты замеров

В таблице 3 показаны медианы процессорного времени. Все значения выражены в миллисекундах на одно умножение матриц; обозначение «опт.» относится к оптимизированному алгоритму Винограда.

Таблица 3 – Процессорное время умножения квадратных матриц, мс

N	Классический	Винограда	Винограда, опт.
32	0,0473	0,0320	0,0145
33	0,0488	0,0351	0,0160
64	0,3906	0,2563	0,1099
65	0,4150	0,2808	0,1160
96	1,3672	0,9766	0,5127
97	1,4648	0,9277	0,3906
128	3,3203	2,2461	1,3184
129	3,3203	2,0508	0,8301
192	12,5000	8,2031	4,8828
193	11,3281	7,0312	3,5156
256	29,6875	18,7500	11,3281
257	28,1250	17,9688	8,5938
384	106,2500	68,7500	39,0625
385	96,8750	59,3750	31,2500
512	410,9375	273,4375	234,3750
513	253,1250	170,3125	76,5625

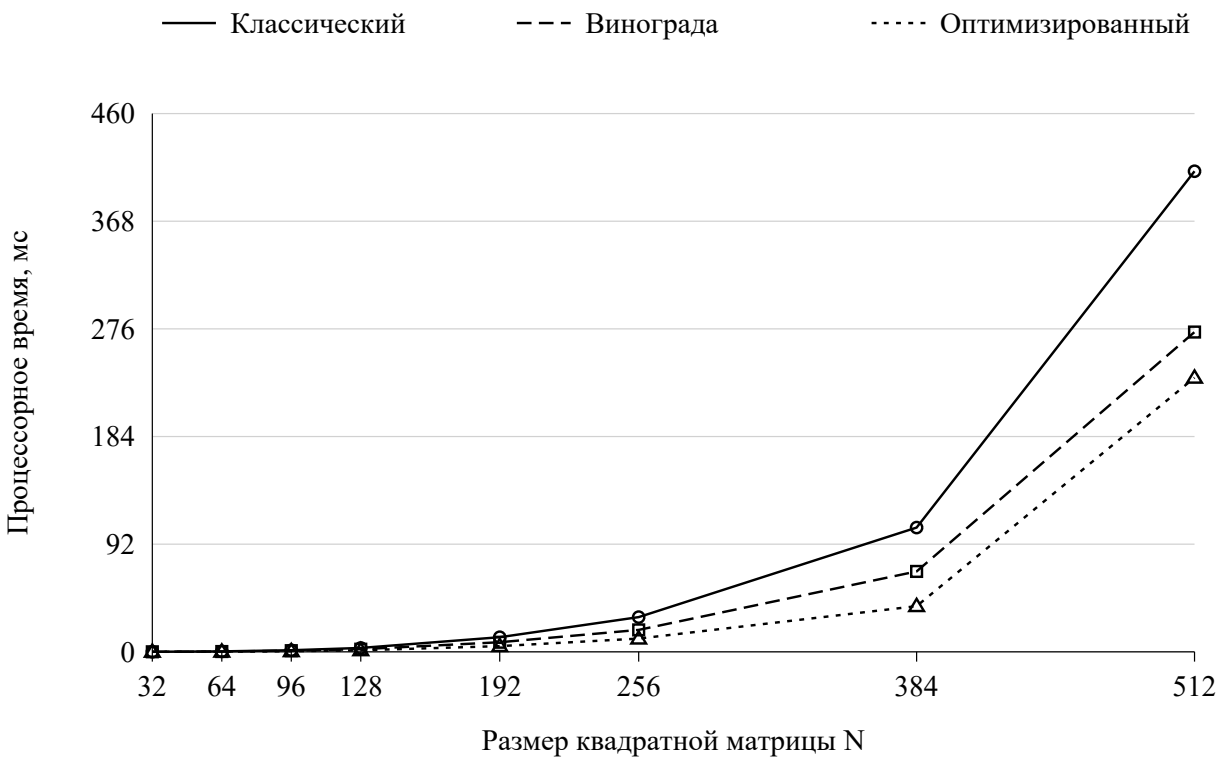


Рисунок 7 – Процессорное время для чётных порядков матриц

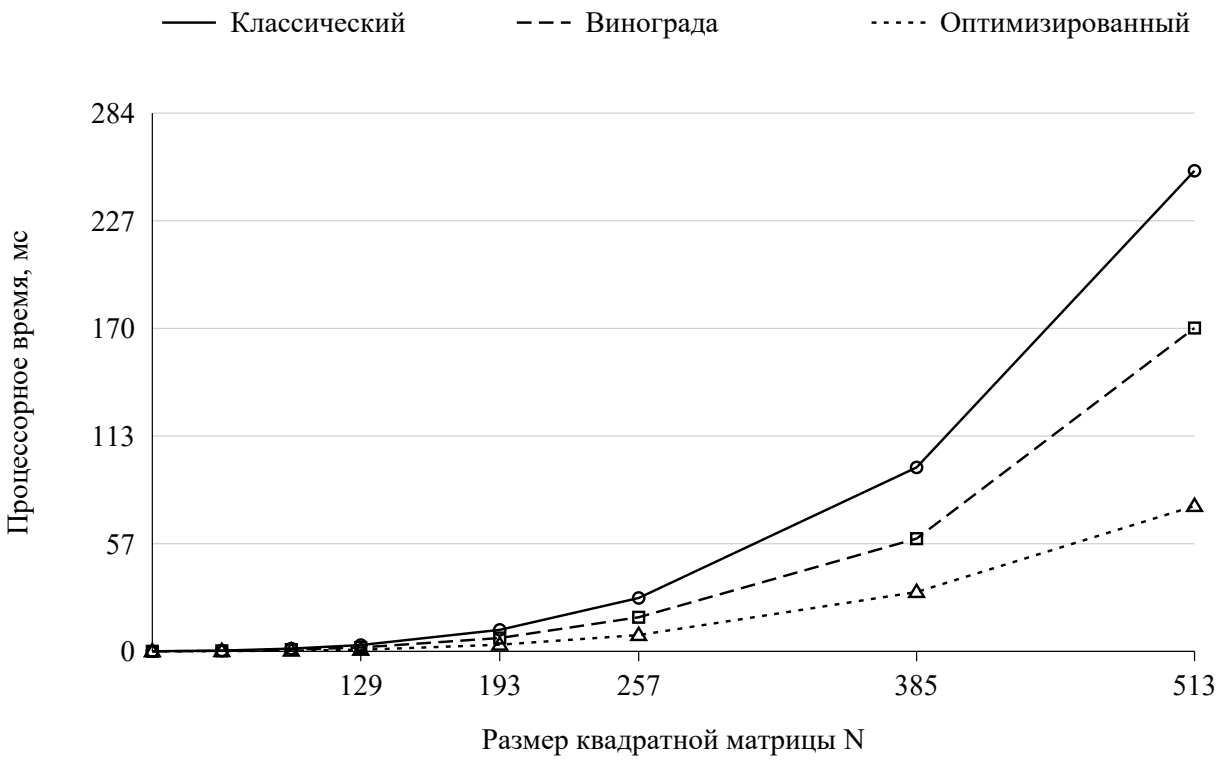


Рисунок 8 – Процессорное время для нечётных порядков матриц

4.4 Сопоставление теории и эксперимента

При $N = 256$ получены времена 29.6875, 18.7500 и 11.3281 мс соответственно. Оптимизированный алгоритм быстрее классического в 2.62 раза и обычного алгоритма Винограда в 1.66 раза. На всех 16 размерах медиана оптимизированного варианта меньше медиан двух других реализаций.

Для количественной проверки порядка роста выполнена линейная аппроксимация $\ln t = p \ln N + b$ по всем измеренным размерам от 64 до 385. Получены показатели $p = 3.10$, $p = 3.06$ и $p = 3.21$ для классического, обычного и оптимизированного алгоритмов соответственно. Диапазон выбран для отдельного анализа до наиболее заметной аномалии на границе 512; все исходные точки сохранены в общей таблице. Оценки показателя по ограниченному диапазону следует рассматривать как эмпирические, без утверждения статистической значимости. На рисунке 9 приведено нормированное время: его изменения показывают, что постоянный коэффициент модели $t \approx cN^3$ зависит от размера и условий выполнения.

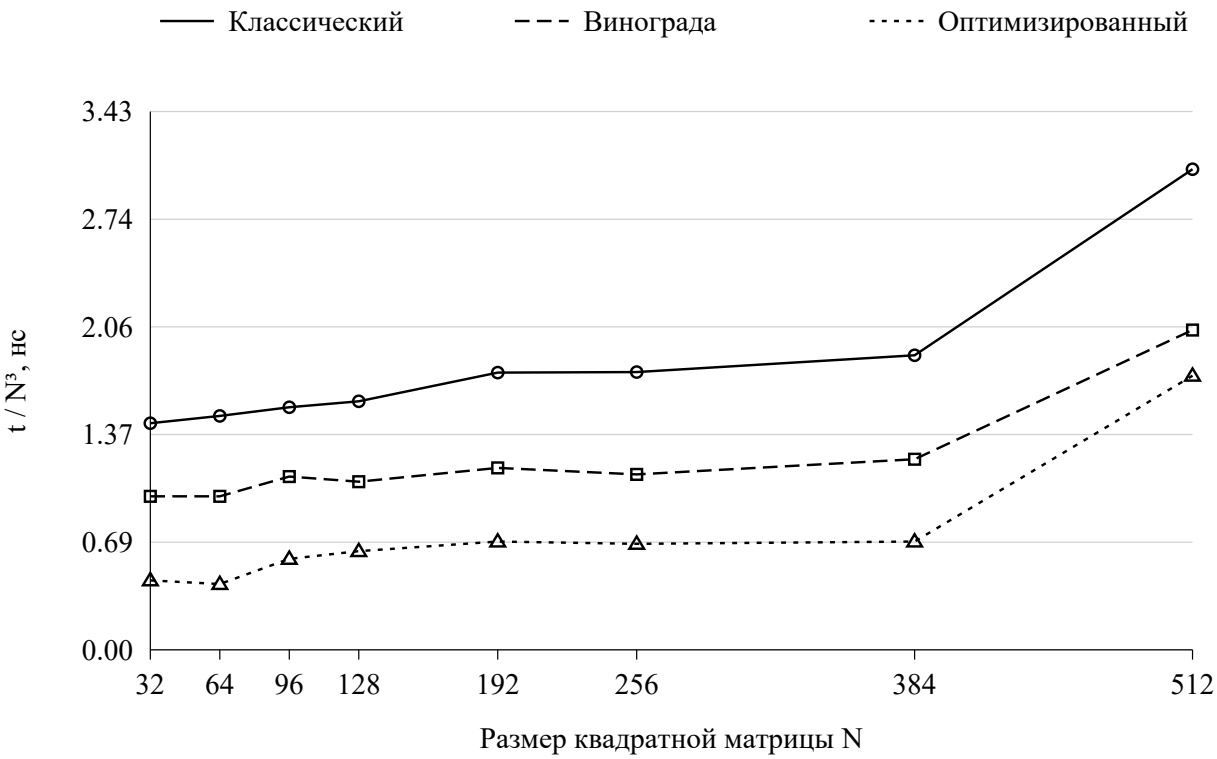


Рисунок 9 – Нормированное время t/N^3 для чётных порядков

При сравнении соседних чётного и нечётного порядков изменяется не только необходимость коррекции, но и число строк и столбцов. Кроме то-

го, меняется шаг доступа к столбцам матрицы B. Поэтому такая пара размеров не изолирует стоимость обработки нечётности. Наблюдаемое отличие результатов $N = 512$ и $N = 513$ согласуется с возможным влиянием кэша и шага доступа; это объяснение является гипотезой, поскольку аппаратные счётчики промахов кэша не измерялись.

Подтверждают ли данные замеров полученные теоретические оценки трудоёмкости? В исследованном диапазоне данные в целом согласуются с кубическим ростом и с ускорением за счёт изменения постоянных множителей. Они не доказывают асимптотическую оценку и не подтверждают точное равенство времени арифметической модели. Уменьшение числа умножений само по себе недостаточно для предсказания времени: существенны обращения к памяти, компилятор и служебные операции. Отдельные отклонения и немонотонность не противоречат $\Theta(N^3)$.

Вывод

На исследованных размерах оптимизированная реализация показала наименьшее процессорное время. Все три алгоритма сохраняют кубический порядок трудоёмкости, а варианты Винограда требуют линейной дополнительной памяти для множителей. Результаты относятся к указанным реализации, компилятору и компьютеру.

ЗАКЛЮЧЕНИЕ

Проведён сравнительный анализ алгоритмов умножения матриц.

В ходе работы описаны классический алгоритм, алгоритм Винограда и его оптимизированный вариант; разработаны схемы; получены формулы трудоёмкости для прямоугольных матриц с учётом чётности общего измерения; оценены затраты памяти; выполнены реализация на C++17, тестирование и массированные замеры процессорного времени.

Все три алгоритма имеют трудоёмкость $\Theta(N^3)$ для квадратных матриц. Алгоритм Винограда сокращает число умножений примерно вдвое в ведущем члене, увеличивая число сложений. Оптимизация сохраняет арифметическую формулу, но изменяет индексацию, накопление сумм и обработку нечётного размера. Классический алгоритм использует $O(1)$ вспомогательной памяти, варианты Винограда — $O(N)$, без учёта входа и результата.

На всех исследованных размерах оптимизированный вариант оказался быстрее двух остальных. Замеры в целом согласуются с теоретическим кубическим порядком роста; конкретные коэффициенты и различия соседних размеров зависят от аппаратных особенностей и компиляции. Цель работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — Введ. 2018-07-01. — URL: https://protect.gost.ru/gost/details/7d280e43-7036-4a69-8e6e-d15867028343 (дата обращения: 23.10.2026).

2. Winograd S. A New Algorithm for Inner Product // IEEE Transactions on Computers. — 1968. — Vol. C-17, no. 7. — P. 693–694. — DOI: https://doi.org/10.1109/TC.1968.227420.

3. GetProcessTimes function (processthreadsapi.h) // Microsoft Learn. — URL: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-getprocesstimes (дата обращения: 23.10.2026).